Конспект 04 — Оперативная память

Содержание

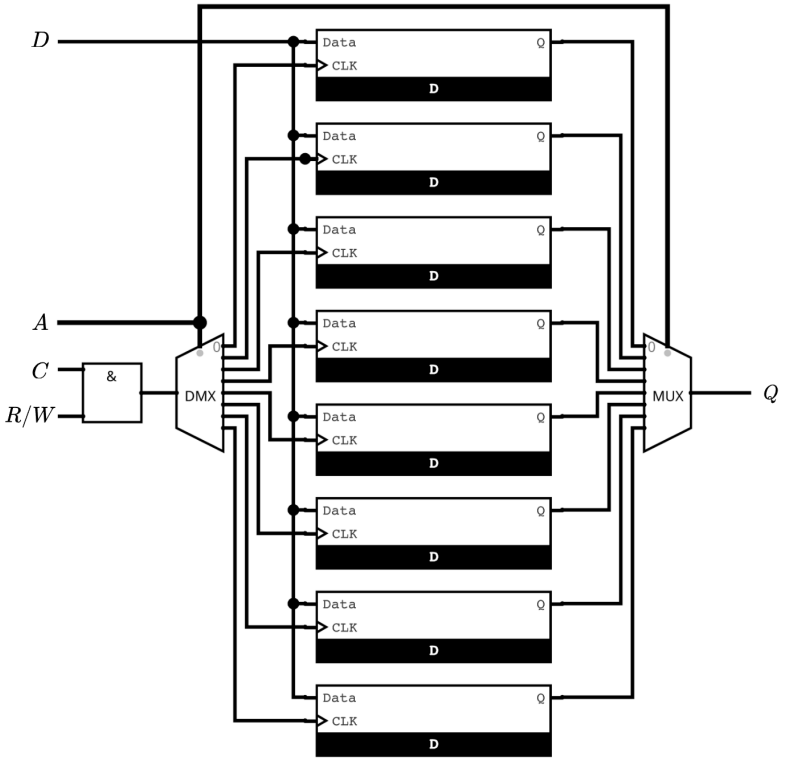
1. Наша первая память
2. Реальные ячейки памяти
    1. Статическая память
    2. Динамическая память
3. Общее устройство оперативной памяти
    1. Матричная структура
    2. Цикл обращения
    3. Тайминги
4. Эволюция оперативной памяти
    1. Первые стандарты
    2. DDR
    3. Графические модули
5. Размещение памяти

”Оно у вас будет работать быстрее, пока будет работать.”
— П. С. Скаков

Наша первая память

Научившись запоминать однобитные значения при помощи триггеров, мы получили всё, что нам необходимо для создания устройства, способного хранить массивы данных произвольного размера. Простейшая его реализация имеет следующий вид:

Схема MEM простейшего модуля памяти на 8 бит



Здесь:

- D — бит данных;
- A — шина адреса;
- C — канал синхронизации;
- R/W — режим работы (*чтение/запись*).

Принцип работы этого модуля памяти достаточно понятен и прост и даже слишком прост, чтобы не иметь изьянов. В частности,

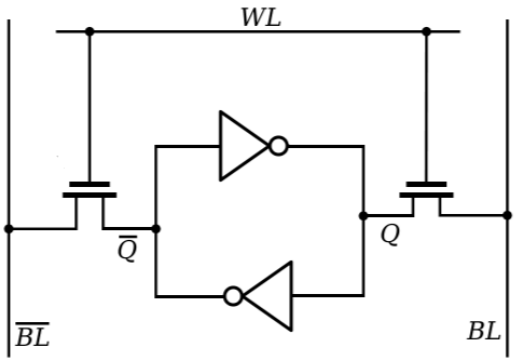
1. наш вход R/W сейчас не имеет особого смысла;
2. мы одновременно используем либо только входы, либо только выходы, то есть половина каналов в любом случае не несет полезной нагрузки;
3. и самое главное: расширять его решительно невозможно, ибо это требует огромного количества проводов и влечет за собой не меньшее количество проблем.

Реальные ячейки памяти

Статическая память

Попробуем придумать что-то получше. Отказавшись, помимо прочего, от использования D-триггеров в силу их сложности (*да, реальная память триггеры в привычном нам виде не использует*), мы можем получить следующую схему:

Ячейка статической памяти (6t SRAM)



На её устройстве стоит остановиться, ибо оно не совсем тривиально или, вернее говоря, совсем нетривиально.

В её основе лежит два циклически соединённых инвертера, которые образуют так называемую *бистабильную защёлку*, которая представляет собой примитивное запоминающее устройство, по принципу работы очень похожее на RS-триггер: выход одного инвертера всегда будет совпадать со входом другого, в результате чего они неизбежно стабилизируются и будут сколь угодно долго сохранять своё состояние.

Управление ей осуществляется через два двунаправленных N-транзистора, которые мы будем назвать *пропускными* или *интерфейсными*. Они, в свою очередь, управляются тремя каналами: WL (*Word Line*), BL (*Bit Line*) и $\overline{\mathrm{BL}}$. Легко видеть, что канал WL отвечает за открытие пропускных транзисторов, и когда на него подаётся 0, то ячейка изолирована от внешнего мира и сохраняет своё значение. Когда на WL подаётся единица, транзисторы открываются, и мы можем взаимодействовать с её содержимым.

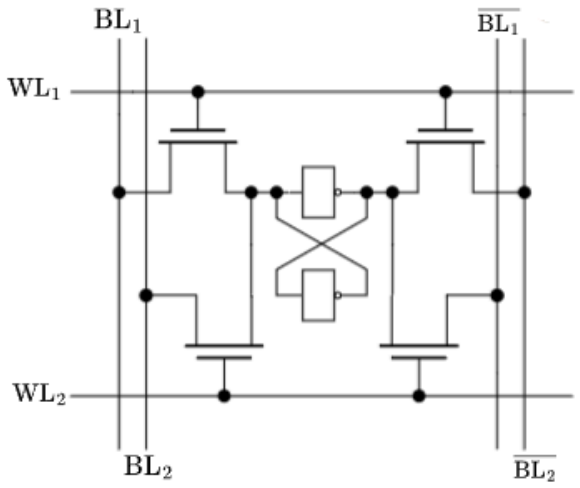
При записи мы открываем транзисторы и подаём целевое значение на BL, а на $\overline{BL}$ подаём его инверсию. Здесь нас интересуют прежде всего ситуация, при которой целевой заряд не совпадает с хранимым, ибо в ином случае провоцировать изменение состояния нечему. Пусть $Q = 1$, $\mathrm{BL} = 0$. В качестве «бутылочного горлышка» в биполярной защёлке выступает P-транзистор, который, как мы помним, в несколько раз менее мощный, чем его негативный брат. Когда мы откроем правый пропускной транзистор, положительный заряд начинает утекать через него по каналу BL быстрее, чем его будут способны поддерживать инвертеры, и когда он утечет полностью, схема стабилизируется, придя в состояние $Q = 0$.
В обратной же ситуации, когда $Q = 0$, $\mathrm{BL} = 1$, ситуация зеркальна: мы не можем пропустить логическую единицу через правый пропускной транзистор, но мы можем «выкачать» единицу из $\overline{Q}$ по каналу $\overline{\mathrm{BL}}$, заряд которого равен 0 (поскольку обратен заряду BL), в результате чего $\overline{Q}$ станет равно 0, а Q, соответственно, станет равно 1, как мы и ожидаем.

Чтение — операция несколько более элегантная. Каналы BL и $\overline{\mathrm{BL}}$ сначала оба заряжаются значением, соответствующим логической единице, после чего единица подаётся также на WL и пропускные трназисторы открываются. В результате этого ячейка начнёт разряжать ту битовую линию, со стороны которой она хранит 0; при этом она служит лишь посредником между положительно заряженным проводом и землёй, то есть, что важно, её собственное значение не теряется. Состояние линий затем отслеживается, и на основании этого делается вывод о текущем хранящемся в ячейке значении.

То, что мы разобрали, называется **шеститранзисторной ячейкой статической памяти**. Важно, что проводов при этом к ней подходит *пять*: помимо трёх управляющих также есть питание и земля.

Следует упомянуть и вариацию на тему статической памяти, носящей название *многопортовой памяти*, которая добавляет к ячейке ещё одну пару интерфейсных транзисторов:

Многопортовая статическая память
(схема вежливо позаимствована с чьего-то конспекта)

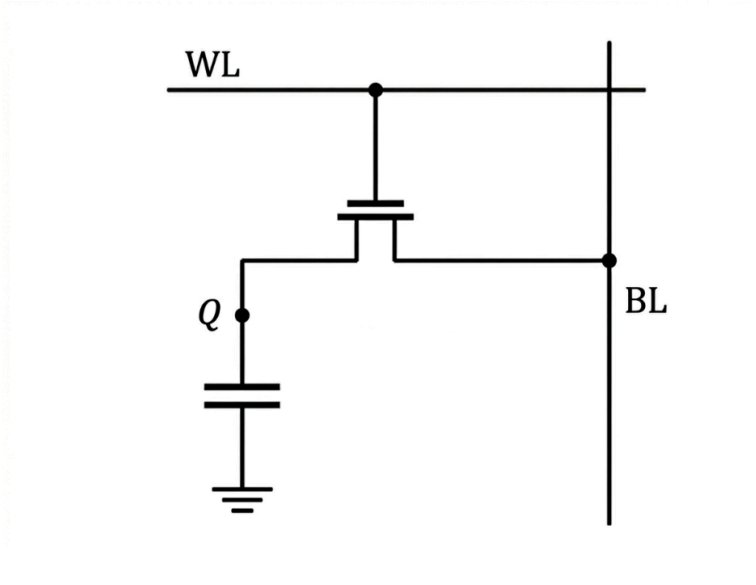


Два набора транзисторов и, как следствие, управляющих каналов, позволяют нам параллельно работать с несколькими ячейками в один момент времени. Подробнее про вопрос эффективности и сопряженных показателей такой схемы мы отдельно будем говорить позже; сейчас лишь стоит отметить, что на практике она встречается редко, потому что достичь того же эффекта иным путём, который мы тоже обсудим.

Динамическая память

Можно ли ещё проще? Да!
Ячейка памяти может использовать лишь *один* транзистор. Выглядит это вот так:

Ячейка динамической памяти
(1t DRAM)



Для взаимодействия с этим устройством точно также, как и в случае статической памяти, нам необходимо подать единичный заряд на WL. В процессе записи мы просто открываем транзистор и подаём целевой сигнал на битовую линию, в результате чего конденсатор либо заряжается от неё, либо разряжается в неё. Для того же, чтобы прочитать значение, нам необходимо зарядить битовую линию ровно до половины напряжения питания и открыть транзистор, что, как и ранее, спровоцирует соответствующее хранимому сигналу изменение напряжения битовой линии.

Драма здесь в том, что конденсатор способен хранить лишь крошечный по своей силе заряд. Первое и менее страшное — из-за этого нам требуется использовать усилители, чтобы суметь зафиксировать изменения, вызванные зарядом ячейки. Второе и более страшное — чтение убивает наши данные, то есть полностью разряжает конденсатор. Более того, этот заряд будет сам со временем утекать. В связи с этим нам нужно постоянно проводить процесс *регенерации*, то есть чтения и перезаписи, чтобы поддерживать память в валидном состоянии.

Это обстоятельство, как и сам факт необходимости заряда конденсатора при каждой записи, ощутимо негативно сказывается на скорости работы такой памяти; это, впрочем, компенсируется её крошечными размерами и общей простотой устройства, что в совокупности позволяет разместить гораздо больше ячеек на одном кристалле сравнительно статической памяти.

Говоря в двух словах, статическая память быстрее, в то время как динамическая — дешевле. Отсюда легко заключить, что DRAM ячейки используются в оперативной памяти, а SRAM — в памяти процессора, будь то кэш или регистры. Может, впрочем, возникнуть идея сделать оперативную память статической, но она, увы, обречена: ощутимого прироста в производительности это не даст, ибо в этой ситуации она упирается не столько в скорость самой памяти, сколько в физическое расстояние от неё до процессора. Обратная же ситуация имеет место: некоторые процессоры делают

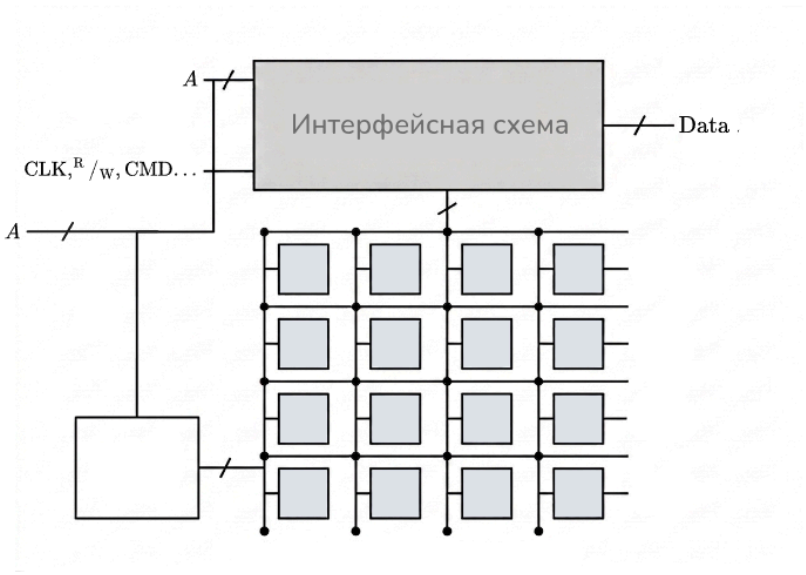
последние уровни кэша динамическими, чтобы увеличить его объем. Практический смысл этих слов станет более ясен, когда мы отдельно обсудим соответствующие темы.

Общее устройство оперативной памяти

Матричная структура

Как статические, так и динамические ячейки объединяются не в линейную, а в прямоугольную структуру, в которой ряды и столбцы разделяют строковые и битовые линии соответственно. В результате этого для, скажем, миллиона бит понадобится уже не миллион проводов, а всего лишь порядка тысячи, что не может не радовать. Получившаяся матрица оснащается *интерфейсной схемой*, через которую осуществляется всё взаимодействие с её содержимым.

Общее устройство модуля памяти

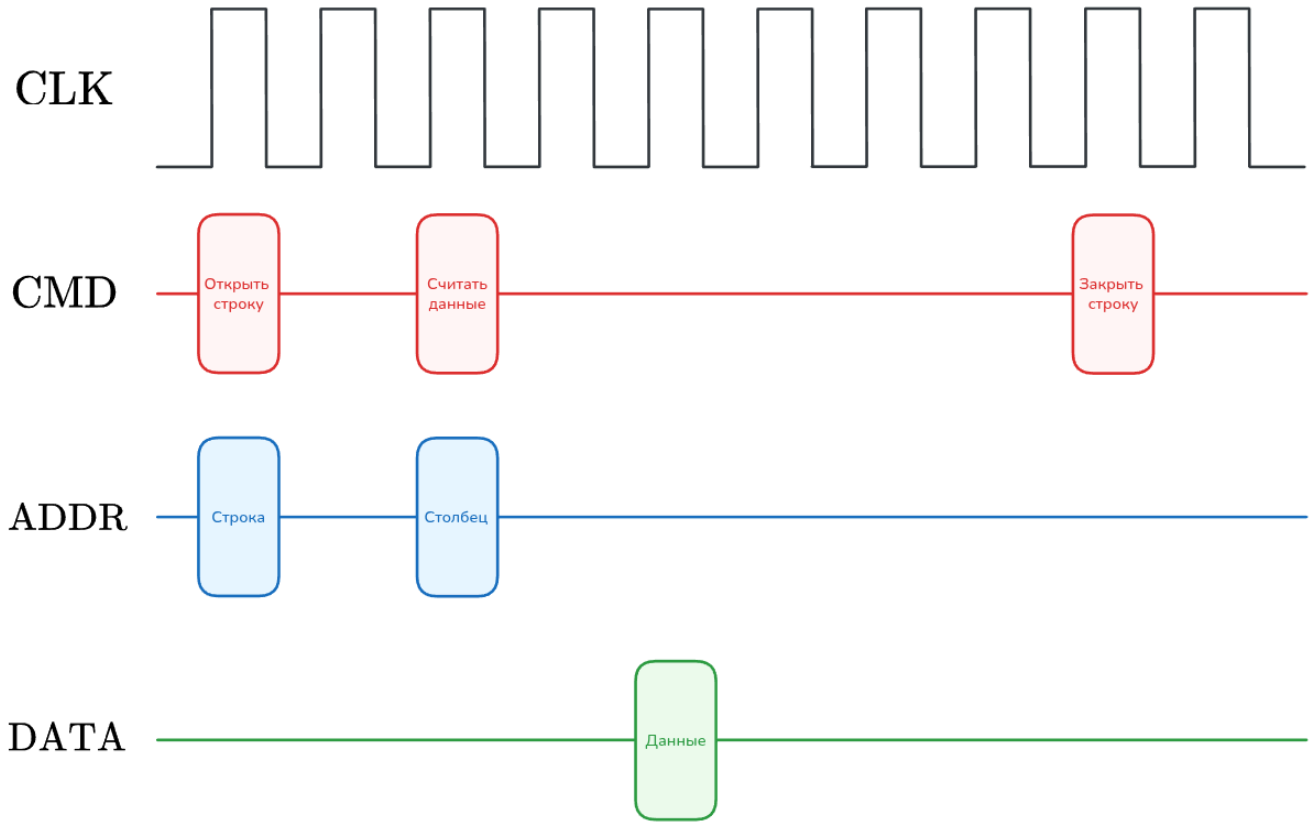


Из такого устройства следует важное замечание: даже если мы хотим обратиться к единственной ячейке из миллиона, мы *обязаны* прочесть всю строку, ибо включение WL неизбежно откроет все транзисторы на ней. Значения битовых линий на всех столбцах, кроме целевого, должны быть выставлены в соответствии с режимом чтения, чтобы не повредить содержимое побочно затронутых ячеек. Запись значений с нужной строки происходит в статический буфер в интерфейсной схеме. Это, впрочем, не совсем негативный эффект, поскольку к отдельным битам мы не обращаемся практически никогда, и вместо этого читаем данные группами по несколько байт.

Следует также отметить, что обычно во внешних модулях памяти выводят наружу лишь половину линий адреса: либо строковые, либо столбцовые, и то, что они означают, определяется командами, посланными памяти.

Цикл обращения

Рассмотрим на примере операции чтения (*запись происходит согласно всем тем же принципам*), как происходит взаимодействие с таким модулем. Последовательность необходимых для неё сигналов выглядит следующим образом:

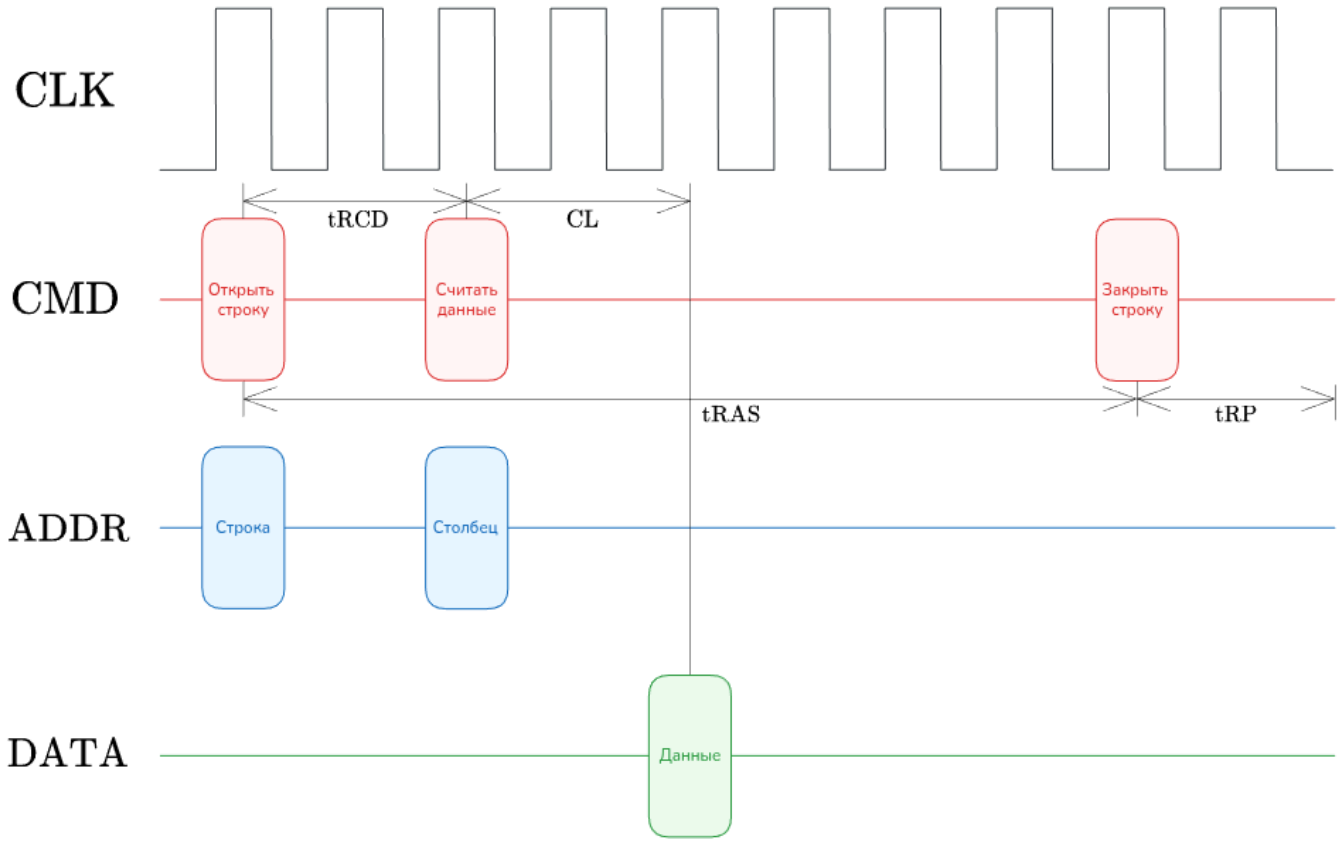


Как мы уже обозначили, чтение начинается с команды открытия строки, одновременно с которой на линию адреса передаётся, собственно, адрес этой самой строки. Спустя какое-то время этот процесс завершается, и мы можем считать данные из конкретной ячейки, указав номер столбца. Спустя ещё некоторое время эти данные появляются на выделенных им линиях. В конце всего этого действа критически важно прочтённую строку обратно закрыть — во-первых потому, что мы больше не хотим воздействовать на хранящиеся там данные при следующих операциях с памятью, а во-вторых потому, что в случае динамической памяти с этой операцией сопряжён процесс регенерации данных: открыв строку, мы не столько скопировали, сколько «переместили»[1] данные в буфер интерфейсной схемы, так что нам, конечно, следует их вернуть.

Тайминги

Примечательно в нашей схеме вот что: внешний модуль памяти, вообще-то, ничего не знает про то, когда закончится каждый из процессов, которые мы рассмотрели, и эту информацию мы должны предоставлять ему сами. Временные промежутки,

определяющие длительность этапов работы с данными в оперативной памяти, называются *тайимингами*. Среди них выделяется четыре основных:



Здесь:

tRCD (*RAS to CAS Delay*)	Задержка между сигналами выбора строки и столбца
CL (*CAS Latency*)	Задержка после выбора столбца
tRAS (*Row Active Time*)	Минимальное время активности строки
tRP (*Row Precharge Time*)	Задержка между закрытием одной строки и открытием другой
RAS (*Row Access Time*)	Время доступа к строке

CAS (*Column Access Time*) Время доступа к столбцу

Набор из этих четырех таймингов — один из ключевых параметров модуля оперативной памяти, поскольку определяют его временные характеристики при самых различных действиях, в чем мы убедимся уже совсем скоро.

#TODO row-hammer

Эволюция оперативной памяти

Первые стандарты

Скажем пару слов о том, какие вообще у памяти можно выделить характеристики.

Самая простая — это *объём*, и с ним всё скорее действительно просто: мы копипастим блок памяти — объём увеличивается, так что на нем мы останавливаться особенно не будем. Что нас интересует ощутимо больше, так это *скорость*, причем, конечно, не какая-то абстрактная, а скорость *чтения* и *записи*. Здесь сразу напрашивается ремарка о том, что, вообще-то, в той системе, которую мы только что рассмотрели, они будут скорее совпадать, чем различаться, в силу схожести самих процессов (хотя, например, для какой-нибудь флеш-памяти это неверно), так что мы будем говорить о скорости *доступа* — времени реакции на запрос, то есть времени от передачи команды до получения первой порции данных — и скорости *передачи* — времени между получением двух последовательных таких порций.

То, что мы рассмотрели выше, исконно определяло классическую модель работы оперативной памяти. Пока скорость работы процессора не превосходила скорость работы памяти, в ней не было изъянов, но со временем поняли, что процессор работает намного быстрее, и память за ним не поспевает, а ускорять её всё равно очень хочется. Из этих соображений люди пришли к ряду стандартов, самые важные из которых мы рассмотрим далее.

> **FPM DRAM** (1987 — 1994)
>
> Ключевым отличием FPM (*Fast Page Mode*) DRAM от рассмотренной стандартной модели служит ускоренная работа с данными, размещёнными в пределах одной строки. Легко заметить, что если два подряд идущих запроса к памяти взаимодействуют с одной и той же строкой, то нет никакого смысла закрывать её на промежуток между этими запросами — эту идею FPM и эксплуатирует. Такая оптимизация неизбежно улучшает скорость передачи и, в случае близко расположенных данных, улучшает также скорость доступа, то есть уже здесь мы

можем заметить, что скорость доступа неравномерна: строго говоря, скорость доступа к разрозненным при достаточно маленькой частоте запросов составит $\mathrm{tRCD} + \mathrm{CL}$ тактов, в то время как при частом обращении доступ потребует $\mathrm{tRAS} + \mathrm{tRP}$ тактов. Такое поведение наследуется всеми последующими стандартами, и на этом этапе следует вынести в первую очередь ту главную практическую мысль, что время обращения к памяти очень сильно зависит от того, как именно это обращение происходит.

EDO DRAM (1994 — 1997)

Здесь стоит сказать, что на самом деле приведенное выше состояние сигналов не совсем отражало реальное положение дел в классической модели RAM. На самом деле во всю длительность операции чтения нам необходимо было держать адрес столбца на соответствующей линии. Стандарт EDO (*Extended Data Output*) решил эту проблему установкой буфера, который копирует это значение, позволяя нам освобождать канал раньше, то есть так, как это было показано изначально. Это хорошо тем, что к моменту окончания операции чтения мы уже можем подготовить адрес следующего столбца, которым будем манипулировать.

BEDO DRAM (1995 — 1997)

Стандарт BEDO (*Burst Extended Data Output*) улучшал скорость передачи путем того, что выдавал на запрос чтения до четырех последовательных порций данных вместо одной. Несмотря на то, что он был быстрее обычного EDO, он использовался не очень широко и в итоге быстро проиграл конкуренцию следующему стандарту.

SDRAM (1996 — 2002)

S здесь любезно позаимствована у слова «синхронизация». К чему это?

Между процессором и памятью (*хотя современные процессоры допускают интеграцию*) есть вспомогательное устройство — *контроллер памяти*, — которое переводит запрос на чтение или запись во все технические подробности взаимодействия с памятью; помимо прочего, именно он знает про стандарт RAM.

Под синхронизацией подразумевается именно что синхронизация самого модуля памяти и контроллера. До этого стандарта они работали асинхронно, то есть имели два независимых тактовых генератора, из-за чего контроллер не мог строго выждать время, которое задаётся тем или иным таймингом, в связи с чем они фактически были больше. Когда же они стали работать синхронно, у нас появилась возможность точно соблюдать все тайминги памяти.

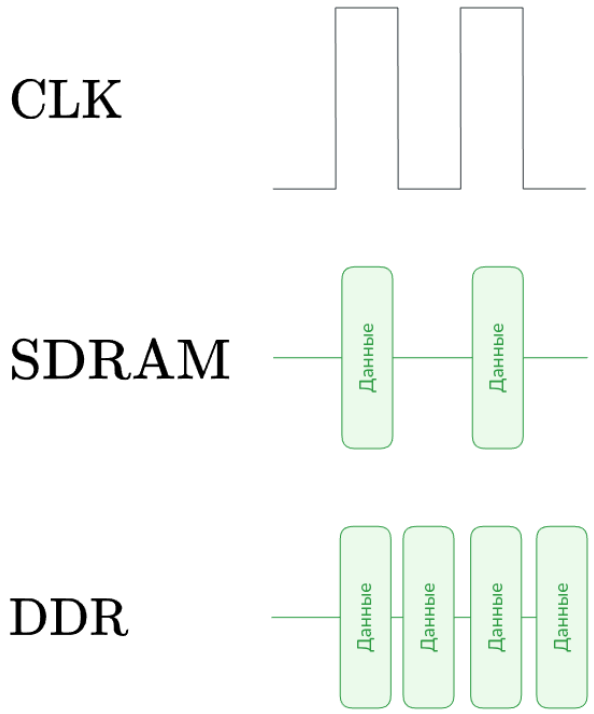
Ещё одно важное изменение, которое принёс этот стандарт, заключается в расширении шины данных до 64 бит. В SDRAM также возникло понятие *банков*

> *памяти*, о которых мы поговорим чуть позже.

DDR

Читателю наверняка известно это название. Мы остановимся на этой серии стандартов отдельно, поскольку они являются устоявшимися на сегодняшний день.

DDR — аббревиатура от *Double Data Rate*. Этот стандарт не вносит никаких изменений в логику работы самого модуля памяти или контроллера: он регулирует лишь работу линии данных. Главная его идея в том, чтобы передавать данные между двумя нашими устройствами дважды за один такт; более строго говоря, данные стали передаваться и на фронте, и на спаде; ещё более строго говоря, в момент пересечения двух противофазных сигналов синхронизации одной частоты:



Тактовая частота контроллера и самого модуля при этом остаются равными, а такое поведение достигается засчет расширения выходной шины модуля памяти: он выдаёт 128 бит единожды за такт, первую половиную отправляет сразу же, а вторую задерживает на полтакта. Весь этот процесс также называется *prefetch* и отличается от *burst* тем, что происходит на одном шаге, а не последовательно.

Скорость доступа в такой схеме остаётся неизменной, однако скорость передачи удваивается. В частности, DDR на частоте 200 МГц достигает такой же скорости передачи, что и SDRAM на частоте 400 МГц, хотя и проигрывает в этом случае в скорости доступа.

Следующим стандартом был DDR2, который удвоил частоту проводящего канала и, соответственно, размер выходной шины модуля памяти. Удивительно, но DDR3 эти

показатели учетверил.

Стоит сказать, что в стандартах изменялось, конечно, далеко не только то, что было отмечено. Например, с каждым новым стандартом улучшалось энергопотребление памяти, и в этом вопросе важна не столько пиковая мощность, ибо она у оперативной памяти откровенно невелика, сколько факт постоянной нагрузки, что особенно критично для мобильных устройств. Вот из такого рода небольших изменений и состоит стандарт DDR4, который является «отшлифованной» версией DDR3.

В стандарте DDR5 снова произошло удвоение частоты, но с некоторой хитростью: единый выходной канал был разделён на два независимых равномощных субканала, что позволило процессору одновременно делать два запроса к одному модулю памяти.

Графические модули

Стандарты DDR стараются соблюдать баланс между скоростью доступа, скоростью передачи, энергоэффективностью и стоимостью. Параллельно с ними развивались стандарты GDDR, которые были ориентированы на графические устройства и оптимизировали в первую очередь скорость передачи, жертвуя всеми остальными параметрами (о том, почему это важно для видеокарт, мы узнаем позже, когда будем говорить про них отдельно). Среди них можно выделить более старые GDDR3 на основе DDR2 и GDDR5 на основе DDR4, GDDR5X с удвоенной тактовой частотой проводящего канала, более новый GDDR6, модуль памяти в котором может работать в режиме QDR (*Quad Data Rate*) и некий GDDR6X, который является не столько самостоятельным стандартом, сколько договоренностью между NVIDIA и Micron насчет использования модуляции PAM4 для передачи данных.

Для графических процессоров также существует независимый от DDR стандарт HBM (*High Bandwidth Memory*), который рассчитан на установку модулей памяти практически вплотную к кристаллу и очень широкую шину данных. Идея в том, чтобы радикально сократить длину проводов и увеличить их количество, снизив тем самым тактовую частоту и энергопотребление, которое является большой проблемой GDDR и графических процессоров в целом. Это прогрессивное и удачное решение, но, увы, до сих пор не слишком распространенное в силу сложности производство и, как следствие, высокой стоимости.

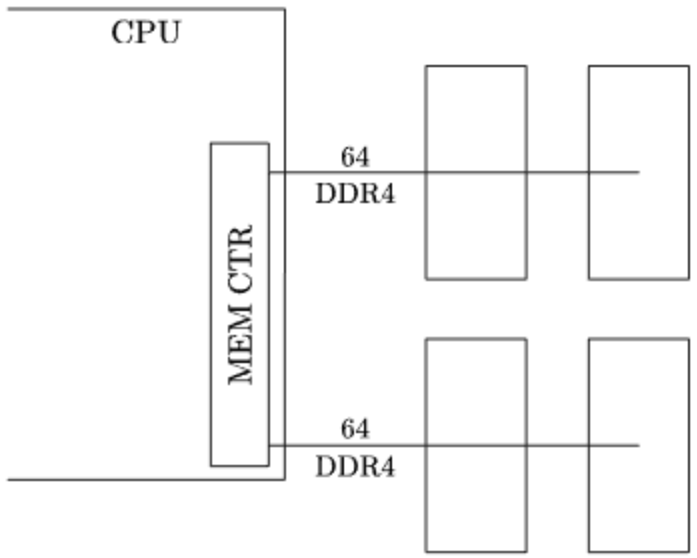
Размещение памяти

Вспомним про две вещи, которые мы упомянули, но очень ловко проигнорировали необходимость их раскрывать: многопортовые ячейки и банки памяти. Было сказано, что эффекта параллельности можно достичь и без усложнения устройства ячейки. Давайте

просто разместим две матрицы рядом друг с другом! Такая организация и называется *многобанковой* — она существенно проще и дешевле, чем многопортовая, а также легче масштабируется, что позволяет допускать даже до четырех одновременных обращений к памяти. Хотя параллельность в её рамках скорее условная: мы можем получить одновременный доступ к двум ячейкам лишь тогда, когда они находятся в разных банках, то есть на разных матрицах. Тем не менее, именно засчет дешевизны она почти всегда предпочитается многопортовой, за исключением разве что регистров процессора, где многопортовая память может эффективно применяться.

Напоследок следует сказать, как оперативная память существует на плате.

В современных системах контроллер памяти встроен в процессор. Что ещё интереснее, он часто двухканальный (*в процессорах покруче каналов побольше*), причём на каждом канале обычно располагается по несколько модулей памяти:



Мотивация здесь следующая. Если установить единственный модуль памяти, то он просто будет работать в соответствии со своей спецификацией. Если последовательно установить два модуля на один канал, то тоже не произойдёт ничего особенного: оба будут работать, их объёмы сложатся, а скорость либо не изменится, либо даже ухудшиться, потому что контроллеру работать с несколькими модулями очевидно сложнее, чем с одним. Но если разместить два очень похожих, в идеале одинаковых, модуля на два различных канала, то контроллер может перейти в двухканальный режиим работы: два 64-битных канала условно объединяются в единый 128-битный, что приводит к уже так полюбившемуся нам двукратному увеличению скорости передачи.

С последовательным размещением связано также такое понятие, как *ранк* — логический блок данных шириной в 64 бита. Многоранковые схемы памяти для контроллера выглядят как несколько «виртуальных» модулей, последовательно расположенные на одном канале. Поскольку ранки независимы друг от друга, иметь их более одного скорее

хорошо благодаря силе параллельности, но по уже упомянутым причинам перебарщивать с их количеством не стоит.

Источники

1. Скаков П. С. — Лекции по аппаратному обеспечению вычислительных систем, 2 семестр, 2026

---

1. см. `std::move` (https://en.cppreference.com/cpp/utility/move) ↩